

Q1. Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.

```

main.py Visualize Run Output
1 MAX_RECORDS = 100
2
3 heap = []
4
5 def insert_record():
6     if len(heap) >= MAX_RECORDS:
7         print("Heap File is full!")
8         return
9
10    record_id = int(input("Enter ID: "))
11    name = input("Enter Name: ")
12
13    heap.append({
14        "id": record_id,
15        "name": name,
16        "deleted": False
17    })
18
19    print("Record inserted successfully.")
20
21
22 def delete_record():
23     record_id = int(input("Enter ID to delete: "))
24
25     for record in heap:
26         if record["id"] == record_id and not record["deleted"]:

```

```

===== Heap File Organization =====
1. Insert Record
2. Delete Record
3. Sequential Scan
4. Exit
Enter your choice: 1
Enter ID: 101
Enter Name: Alice
Record inserted successfully.

===== Heap File Organization =====
1. Insert Record
2. Delete Record
3. Sequential Scan
4. Exit
Enter your choice: 1
Enter ID: 102
Enter Name: Bob
Record inserted successfully.

===== Heap File Organization =====
1. Insert Record
2. Delete Record
3. Sequential Scan
4. Exit

```

Activate Windows
Go to Settings to activate Windows.

Q2. Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.

```

main.py Visualize Run Output
1 TABLE_SIZE = 100
2
3 hash_table = [[] for _ in range(TABLE_SIZE)]
4
5 def hash_function(student_id):
6     return student_id % TABLE_SIZE
7
8 def insert(student_id, name, department):
9     index = hash_function(student_id)
10    hash_table[index].append((student_id, name, department))
11
12 def search(student_id):
13     index = hash_function(student_id)
14
15     for student in hash_table[index]:
16         if student[0] == student_id:
17             print("ID:", student[0])
18             print("Name:", student[1])
19             print("Department:", student[2])
20             return
21
22     print("Student not found")
23
24 def display():
25     for i in range(TABLE_SIZE):
26         if hash_table[i]:

```

```

'CSE'), (489, 'Student489', 'CSE'))
90 : [(90, 'Student90', 'CSE'), (190, 'Student190', 'CSE'), (290, 'Student290', 'CSE'), (390, 'Student390', 'CSE'), (490, 'Student490', 'CSE')]
91 : [(91, 'Student91', 'CSE'), (191, 'Student191', 'CSE'), (291, 'Student291', 'CSE'), (391, 'Student391', 'CSE'), (491, 'Student491', 'CSE')]
92 : [(92, 'Student92', 'CSE'), (192, 'Student192', 'CSE'), (292, 'Student292', 'CSE'), (392, 'Student392', 'CSE'), (492, 'Student492', 'CSE')]
93 : [(93, 'Student93', 'CSE'), (193, 'Student193', 'CSE'), (293, 'Student293', 'CSE'), (393, 'Student393', 'CSE'), (493, 'Student493', 'CSE')]
94 : [(94, 'Student94', 'CSE'), (194, 'Student194', 'CSE'), (294, 'Student294', 'CSE'), (394, 'Student394', 'CSE'), (494, 'Student494', 'CSE')]
95 : [(95, 'Student95', 'CSE'), (195, 'Student195', 'CSE'), (295, 'Student295', 'CSE'), (395, 'Student395', 'CSE'), (495, 'Student495', 'CSE')]
96 : [(96, 'Student96', 'CSE'), (196, 'Student196', 'CSE'), (296, 'Student296', 'CSE'), (396, 'Student396', 'CSE'), (496, 'Student496', 'CSE')]
97 : [(97, 'Student97', 'CSE'), (197, 'Student197', 'CSE'), (297, 'Student297', 'CSE'), (397, 'Student397', 'CSE'), (497, 'Student497', 'CSE')]
98 : [(98, 'Student98', 'CSE'), (198, 'Student198', 'CSE'), (298, 'Student298', 'CSE'), (398, 'Student398', 'CSE'), (498, 'Student498', 'CSE')]
99 : [(99, 'Student99', 'CSE'), (199, 'Student199', 'CSE'), (299, 'Student299', 'CSE'), (399, 'Student399', 'CSE'), (499, 'Student499', 'CSE')]

Search Result:
ID: 250
Name: Student250
Department: CSE

```

Q3. Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.

main.py	Visualize	Run	Output
<pre> 1 records = [2 (101, "Alice"), 3 (102, "Bob"), 4 (103, "Charlie"), 5 (104, "David"), 6 (105, "Eve"), 7 (106, "Frank"), 8 (107, "Grace"), 9 (108, "Harry"), 10 (109, "Ivy"), 11 (110, "Jack") 12] 13 14 def binary_search(key): 15 low = 0 16 high = len(records) - 1 17 18 while low <= high: 19 mid = (low + high) // 2 20 21 if records[mid][0] == key: 22 print("Record Found") 23 print("ID:", records[mid][0]) 24 print("Name:", records[mid][1]) 25 return </pre>			<pre> Sorted File: (101, 'Alice') (102, 'Bob') (103, 'Charlie') (104, 'David') (105, 'Eve') (106, 'Frank') (107, 'Grace') (108, 'Harry') (109, 'Ivy') (110, 'Jack') Enter Primary Key to search: 106 Record Found ID: 106 Name: Frank === Code Execution Successful === </pre>

Q4. Implement a B-Tree of order 4 supporting insert, search, and display operations. Insert the keys: 10, 20, 5, 6, 12, 30, 7, 17.

main.py	Visualize	Run	Output
<pre> 8 class BTree: 9 def display(self, node=None, level=0): 10 print("Level", level, ":", node.keys) 11 12 if not node.leaf: 13 for child in node.children: 14 self.display(child, level + 1) 15 16 # Create B-Tree 17 btree = BTree() 18 19 # Insert keys 20 keys = [10, 20, 5, 6, 12, 30, 7, 17] 21 22 for key in keys: 23 btree.insert(key) 24 25 print("B-Tree:") 26 btree.display() 27 28 # Search 29 key = int(input("Enter key to search: ")) 30 31 if btree.search(btree.root, key): 32 print("Key found") </pre>			<pre> B-Tree: Level 0 : [10, 20] Level 1 : [5, 6, 7] Level 1 : [12, 17] Level 1 : [30] Enter key to search: 25 Key not found === Code Execution Successful === </pre>

Q5. Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys.

```
main.py Visualize Run Output
8 class BTree:
81     def display(self, node=None, level=0):
85         print("Level", level, ":", node.keys)
86
87         if not node.leaf:
88             for child in node.children:
89                 self.display(child, level + 1)
90
91
92 # Create B-Tree
93 btree = BTree()
94
95 # Insert keys
96 keys = [10, 20, 5, 6, 12, 30, 7, 17]
97
98 for key in keys:
99     btree.insert(key)
100
101 print("B-Tree:")
102 btree.display()
103
104 # Search
105 key = int(input("\nEnter key to search: "))
106
107 if btree.search(btree.root, key):
108     print("Key found")
```

B-Tree:
Level 0 : [10, 20]
Level 1 : [5, 6, 7]
Level 1 : [12, 17]
Level 1 : [30]

Enter key to search: 25
Key not found

=== Code Execution Successful ===

Q6. Write a code to implement a dense primary index on a sorted file. Support search by index key and display the index table.

```
main.py Visualize Run Output
1 records = [
2     (101, "Alice"),
3     (102, "Bob"),
4     (103, "Charlie"),
5     (104, "David"),
6     (105, "Eve"),
7     (106, "Frank"),
8     (107, "Grace"),
9     (108, "Harry")
10 ]
11
12 # Create dense index
13 index = []
14
15 for i in range(len(records)):
16     index.append((records[i][0], i))
17
18
19 def display_index():
20     print("\nIndex Table")
21     print("Key\tPosition")
22
23     for key, position in index:
24         print(key, "\t", position)
25
```

Index Table
Key Position
101 0
102 1
103 2
104 3
105 4
106 5
107 6
108 7

Enter index key to search: 105

Record Found
Key: 105
Name: Eve

=== Code Execution Successful ===

Q7. Implement extendible hashing in code. Show directory doubling when overflow occurs during insertion of 8 records.

```

main.py Visualize Run Output
1 BUCKET_SIZE = 2
2
3 class Bucket:
4     def __init__(self, depth=1):
5         self.depth = depth
6         self.records = []
7
8
9 global_depth = 1
10
11 bucket0 = Bucket(1)
12 bucket1 = Bucket(1)
13
14 directory = [bucket0, bucket1]
15
16
17 def hash_function(key):
18     return key
19
20
21 def insert(key):
22     global global_depth, directory
23
24     while True:
25         index = hash_function(key) & ((1 << global_depth) - 1)
26         bucket = directory[index]
27
28         if len(bucket.records) < BUCKET_SIZE:
29             bucket.records.append(key)
30             return
31
32         # Overflow occurred, double the directory
33         global_depth += 1
34         new_directory = []
35         for i in range(0, len(directory), 2):
36             old_bucket0 = directory[i]
37             old_bucket1 = directory[i+1]
38             new_bucket0 = Bucket(global_depth)
39             new_bucket1 = Bucket(global_depth)
40             new_bucket0.records = old_bucket0.records
41             new_bucket1.records = old_bucket1.records
42             new_directory.append(new_bucket0)
43             new_directory.append(new_bucket1)
44         directory = new_directory
45
46         # Re-insert the key
47         insert(key)
48
49
50 # Insert 8 records
51 for key in range(1, 9):
52     insert(key)
53
54 # Print the final directory
55 print("Final Directory:")
56 for i in range(0, len(directory), 2):
57     bucket0 = directory[i]
58     bucket1 = directory[i+1]
59     print(f"{i} -> {bucket0.records} Local Depth: {bucket0.depth}")
60     print(f"{i+1} -> {bucket1.records} Local Depth: {bucket1.depth}")
61
62
63 # Output
64 Inserted: 1
65 Inserted: 3
66
67 Overflow occurred for key: 5
68 Directory doubling...
69 Global Depth: 2
70 Inserted: 5
71 Inserted: 7
72 Inserted: 2
73 Inserted: 4
74
75 Overflow occurred for key: 6
76 Directory doubling...
77 Global Depth: 3
78 Inserted: 6
79 Inserted: 8
80
81 Directory:
82 0 -> [4, 8] Local Depth: 2
83 1 -> [1, 5] Local Depth: 2
84 2 -> [2, 6] Local Depth: 2
85 3 -> [3, 7] Local Depth: 2
86 4 -> [4, 8] Local Depth: 2
87 5 -> [1, 5] Local Depth: 2
88 6 -> [2, 6] Local Depth: 2

```

Q8. Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval.

```

main.py Visualize Run Output
1 records = list(range(1, 101))
2 block_size = 10
3
4 blocks = []
5 for i in range(0, len(records), block_size):
6     blocks.append(records[i:i + block_size])
7
8 def sequential_retrieval(key):
9     io = 0
10
11     for block in blocks:
12         io += 1
13
14         if key in block:
15             print("Sequential Retrieval:")
16             print("Record Found:", key)
17             print("I/O Operations:", io)
18             return io
19
20     print("Record Not Found")
21     return io
22
23
24 def indexed_retrieval(key):
25     index = {}
26
27     for i, block in enumerate(blocks):
28         for record in block:
29             index[record] = i
30
31     if key in index:
32         io = 1
33         return io
34     else:
35         print("Record Not Found")
36         return 0
37
38
39 # Output
40 Total Records: 100
41 Block Size: 10
42 Total Blocks: 10
43
44 Enter record to retrieve: 87
45 Sequential Retrieval:
46 Record Found: 87
47 I/O Operations: 9
48
49 Indexed Retrieval:
50 Record Found: 87
51 I/O Operations: 2
52
53 Comparison:
54 Sequential I/O: 9
55 Indexed I/O: 2
56
57 == Code Execution Successful ==

```

Q9. Implement a sparse index on a sorted file in code and compare its search performance with a dense index using timing analysis.

```
main.py Visualize Run Output
1 import time
2
3 records = [(i, "Student" + str(i)) for i in range(1, 10001)]
4
5 dense_index = [(record[0], i) for i, record in enumerate(records)]
6
7 sparse_index = [(records[i][0], i) for i in range(0, len(records), 10)]
8
9
10 def dense_search(key):
11     low = 0
12     high = len(dense_index) - 1
13
14     while low <= high:
15         mid = (low + high) // 2
16
17         if dense_index[mid][0] == key:
18             return records[dense_index[mid][1]]
19
20         if dense_index[mid][0] < key:
21             low = mid + 1
22         else:
23             high = mid - 1
24
25     return None
```

Enter key to search: 8750

Dense Index:
Record: (8750, 'Student8750')
Time: 1.416000031895237e-05 seconds

Sparse Index:
Record: (8750, 'Student8750')
Time: 1.507999684530776e-05 seconds

Index Comparison:
Dense Index Entries: 10000
Sparse Index Entries: 1000
Dense Search Time: 1.416000031895237e-05 seconds
Sparse Search Time: 1.507999684530776e-05 seconds

=== Code Execution Successful ===

Q10. Write a program to construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45, showing all matching records.

```
main.py Visualize Run Output
1 class Node:
2     def __init__(self, leaf=False):
3         self.leaf = leaf
4         self.keys = []
5         self.children = []
6         self.next = None
7
8
9 class BPlusTree:
10     def __init__(self, order=4):
11         self.order = order
12         self.root = Node(True)
13
14     def insert(self, key):
15         if self.root.leaf:
16             if key not in self.root.keys:
17                 self.root.keys.append(key)
18                 self.root.keys.sort()
19
20             if len(self.root.keys) >= self.order:
21                 self.split_root()
22
23     def split_root(self):
24         keys = self.root.keys
25         mid = len(keys) // 2
```

B+ Tree Leaf Nodes:
[10, 20] -> [30, 40] -> None

Range Query (15 to 45):
20 30 40

=== Code Execution Successful ===